# Schriftlicher Testplan: Datenbank-Anbindung (US #43)

| Testfall / Szenario | Was testen wir? (Komponente & Input) | Warum testen wir es? (Testziel & Motivation) | Wie testen wir es? (Testmethode & Vorgehen) |
|---|---|---|---|
| **Round-Trip Test** | Die vollständige Speicherung und das Auslesen eines Test-Datensatzes der Entity `Gründungsidee` mit allen Pflichtfeldern: `Idee_ID`, `gründer_user_ID`, `Beschreibung`, `Position`, `Anforderungen`, `Arbeitsbedingungen`. | Die `Gründungsidee` ist die zentrale Entity der Plattform. Wenn diese nicht korrekt gespeichert und ausgelesen wird, können Gründer keine Ideen veröffentlichen und Studenten können sich nicht bewerben — die gesamte Kernfunktion bricht zusammen. | Integrationstest (Round-Trip-Verfahren, C-R-Ass-D) im Spring-Boot-Kontext:<br><br>**1. C (Create):** Erzeugen eines `Gründungsidee`-Objekts und Speichern via z.B. [GruendungsideeRepository] `.save(idee)` [JPA]<br><br>**2. R (Read):** Auslesen anhand der auto-generierten `Idee_ID` via `findById(id)` [JPA], geschützt durch ein `Optional` [Java]<br><br>**3. Ass (Assert):** Feldweiser Vergleich via `assertEquals()` [JUnit 5] sowie Prüfung auf unterschiedliche Speicheradressen via `assertNotSame()` [JUnit 5]<br><br>**4. D (Delete):** Löschen via `deleteById(id)` [JPA] und Verifikation mit `assertFalse(existsById(id))` [JUnit 5 / JPA] |

| Testfall / Szenario | Was testen wir? (Komponente & Input) | Warum testen wir es? (Testziel & Motivation) | Wie testen wir es? (Testmethode & Vorgehen) |
|---|---|---|---|
| **2. DB-Verbindungstest** | Die Fähigkeit der Anwendung, beim Start erfolgreich eine Verbindung zur PostgreSQL-Datenbank aufzubauen, ohne mit einer Fehlermeldung (z.B. `PSQLException`) abzubrechen — bevor eigene Tabellen (`gründungsidee`, `benutzer` usw.) in der DB existieren. | Ohne funktionierende DB-Verbindung ist die gesamte Plattform nicht lauffähig. Kein Benutzer kann angelegt werden, keine Gründungsidee veröffentlicht, keine Bewerbung eingereicht. Dieser Test stellt sicher, dass VPN, Zugangsdaten und `application.properties` korrekt konfiguriert sind. | Integrationstest im Spring-Boot-Kontext:<br>**1.** Spring startet den Anwendungskontext via `@SpringBootTest` [Spring Boot Test]<br>**2.** Ein Repository (z.B. [BenutzerRepository]) wird per `@Autowired` [Spring] injiziert<br>**3.** Einfache, nicht-verändernde DB-Abfrage via `count()` [JPA]<br>**4.** Prüfung via `assertDoesNotThrow()` [JUnit 5] (keine Exception) und `assertNotNull()` [JUnit 5] (Repository geladen) |